Implementing TPC-H on a Graph-Native Language: A Jac Case Study

Abstract

TPC-H is the industry-standard benchmark for evaluating analytical query processing, traditionally implemented against relational database systems using SQL. This paper presents a complete implementation of all 22 TPC-H queries as graph walker traversals in Jac, a data-spatial programming language in which mobile agents called *walkers* carry computation to data rather than pulling data to computation. We map the eight TPC-H tables to typed graph nodes, foreign keys to directed edges, and each query to one or more walker definitions that traverse the resulting graph. A multi-hop traversal syntax allows walkers to jump directly to target nodes several hops away, reducing per-query code from five or more entry abilities to one or two. All 22 queries are verified against reference SQL answers generated at scale factor 0.01.

1 Introduction

The TPC-H benchmark [1] defines a decision-support workload comprising eight base tables and 22 analytical queries that exercise joins, aggregations, subqueries, and sorting. It has served as the de facto standard for comparing relational database systems for over two decades.

Graph databases and graph-native programming languages offer a fundamentally different computation model: data is stored as nodes and edges, and queries are expressed as traversals rather than relational joins. *Jac* [2] is a data-spatial programming language built on top of Python that introduces the concept of *walkers*—mobile computational agents that move across a graph, executing abilities as they visit each node.

This paper investigates whether a graph-native language can express the full suite of TPC-H queries. We implement all 22 queries in Jac using multi-hop walker traversals, verify correctness against SQL reference answers, and discuss the patterns and trade-offs that emerge from this exercise.

2 Background

2.1 The TPC-H Benchmark

TPC-H models a wholesale supplier environment with eight tables: Region, Nation, Supplier, Part, PartSupp, Customer, Orders, and LineItem. Foreign keys define a hierarchy of relationships. The 22 queries range from simple single-table aggregations (Q1, Q6) to complex multi-table joins with correlated subqueries (Q2, Q21). A *scale factor* (SF) parameter controls the dataset size; SF = 0.01 produces roughly 86,000 rows across all tables.

2.2 The Jac Programming Language

Jac extends Python with first-class graph primitives. The key abstractions are:

Nodes. Typed graph vertices with named attributes, declared with the `node` keyword:

```
node Region {
    has r_regionkey: int,
        r_name: str,
        r_comment: str;
}
```

Edges. Typed directed edges connecting nodes, declared with the `edge` keyword. An edge may carry its own attributes, though in TPC-H the edges are attribute-free labels that encode foreign-key relationships:

```
edge region_has_nation {}
edge nation_has_customer {}
```

Walkers. Mobile computational agents that traverse the graph. A walker has `has` variables (persistent state carried across hops), and *abilities* that fire when the walker enters or exits a particular node type. Inside an ability, the `here` keyword refers to the current node and `self` refers to the walker's own state. The `visit` statement directs the walker to move to other nodes, and `report` returns a result to the caller.

Multi-hop traversal. Jac supports chained edge–node filters in a single `visit` statement:

```
visit [-->(?:Region)->:region_has_nation:->(?:Nation)
       ->:nation_has_customer:->(?:Customer)];
```

This causes the walker to jump directly to the Customer nodes, traversing Region and Nation silently—entry abilities fire *only* on the final target nodes.

Graph construction. The operators `++>` and `+>: EdgeType():+>` create nodes and typed edges, respectively.

3 Graph Schema Design

We map each TPC-H table to a node type and each foreign-key relationship to a directed edge type. Figure 1 shows the resulting schema.

The hierarchy has two principal branches rooted at `root`:

- Region → Nation → Customer → Order → LineItem (with a fork at Nation → Supplier), and
- Part → PartSupp.

The LineItem–PartSupp edge acts as a cross-reference, analogous to a composite foreign key in the relational schema. Each PartSupp node also receives an incoming edge from the relevant Supplier node.

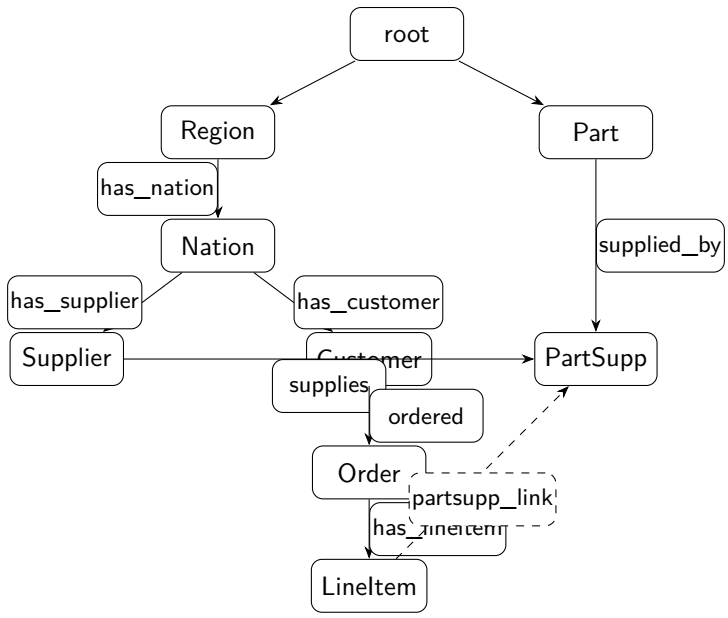

Figure 1: Graph schema for TPC-H. Solid arrows represent the primary traversal hierarchy; the dashed arrow is a cross-reference from LineItem to PartSupp. Only Region and Part connect directly to `root`.

4 Query Implementation Patterns

The 22 TPC-H queries, when expressed as walker traversals, fall into a small number of recurring patterns.

4.1 Direct Multi-Hop Aggregation

Queries that aggregate over LineItem data without needing intermediate node attributes can be expressed with a single multi-hop `visit` from root to lineitem. Q1 (Pricing Summary) is the canonical example:

```
walker Q1_PricingSummary {
    has delta: int = 90;
    has agg: dict = {};
    has cutoff: str = "";

    can on_root with Root entry {
        import datetime;
        self.cutoff = (datetime.date(1998, 12, 1)
            - datetime.timedelta(days=self.delta)).isoformat();
        visit [-->(?:Region)->:region_has_nation:->(?:Nation)
            ->:nation_has_customer:->(?:Customer)
            ->:customer_ordered:->(?:Order)
            ->:order_has_lineitem:->(?:LineItem)];
    }
    can on_lineitem with LineItem entry {
        if here.l_shipdate <= self.cutoff {
            # ... aggregate into self.agg ...
        }
```

```
    }
    can finish with Root exit {
        # ... sort and report results ...
    }
}
```

The walker starts at `root`, jumps directly to every LineItem node (five hops away), and aggregates matching rows. The `Root exit` ability fires after all visited nodes have been processed, serving as the finalization step. Queries Q6, Q14, Q15, Q17, Q19, and Q20 follow this same pattern.

4.2 Filtered Intermediate Traversal

When a query filters on attributes of intermediate nodes, the walker must stop at those nodes, apply the filter, and conditionally continue. Q3 (Shipping Priority) demonstrates this pattern:

```
walker Q3_ShippingPriority {
    has segment: str = "BUILDING",
        date: str = "1995-03-15";
    has order_map: dict = {};
    has agg: dict = {};

    can on_root with Root entry {
        visit [-->(?:Region)->:region_has_nation:->(?:Nation)
            ->:nation_has_customer:->(?:Customer)];
    }
    can on_customer with Customer entry {
        if here.c_mktsegment == self.segment {
            visit [->:customer_ordered:->(?:Order)];
        }
    }
    can on_order with Order entry {
        if here.o_orderdate < self.date {
            self.order_map[here.o_orderkey] = { ... };
            visit [->:order_has_lineitem:->(?:LineItem)];
        }
    }
    can on_lineitem with LineItem entry {
        # ... compute revenue using order_map ...
    }
}
```

The multi-hop visit from root lands the walker on Customer nodes. The `on_customer` ability checks the market segment and only continues the traversal for matching customers. Similarly, `on_order` filters by date before visiting line items. Queries Q4, Q5, Q7, Q8, Q9, Q10, Q12, Q18, and Q21 use variations of this pattern.

4.3 Helper Walker Pattern

Some queries require information that spans both branches of the graph (e.g., joining LineItem data with Supplier nation information). Since a walker cannot simultaneously traverse two branches, we use *helper walkers* that pre-build lookup maps:

```
walker _BuildSuppNationMap {
```

```
    has result: dict = {};

    can on_root with Root entry {
        visit [-->(?:Region)->:region_has_nation:->(?:Nation)];
    }
    can on_nation with Nation entry {
        for supp in [->:nation_has_supplier:->](?:Supplier) {
            self.result[supp.s_suppkey] = here.n_name;
        }
    }
}
```

The main query walker spawns the helper, extracts its `result` map, and uses it during its own traversal:

```
can on_root with Root entry {
    sn_helper = root spawn _BuildSuppNationMap();
    self.supp_nation = sn_helper.result;
    visit [...];
}
```

This pattern is the graph-traversal analog of a hash join in relational processing. Queries Q2, Q5, Q7, Q8, Q9, Q11, Q15, Q16, Q19, and Q20 use one or more helper walkers.

4.4 Two-Phase Traversal

Q20 (Potential Part Promotion) requires data from the LineItem subtree to determine thresholds before querying the Supplier–PartSupp subtree. The walker performs a first-phase traversal to collect line-item quantities, then spawns a second-phase walker with the collected data:

```
can finish with Root exit {
    q20_helper = root spawn _Q20Phase2(
        nation_name=self.nation_name,
        matching_partkeys=self.matching_partkeys,
        li_qtys=self.li_qtys
    );
    report q20_helper.result;
}
```

This pattern naturally arises for queries with correlated subqueries that reference aggregates over other parts of the schema.

4.5 LEFT JOIN Emulation

Q13 (Customer Distribution) requires a LEFT OUTER JOIN between Customer and Orders. In the walker model, we initialize every customer's order count to zero and then increment it for matching orders:

```
can on_customer with Customer entry {
    if here.c_custkey not in self.cust_order_count {
        self.cust_order_count[here.c_custkey] = 0;
    }
    visit [->:customer_ordered:->(?:Order)];
```

```
}
can on_order with Order entry {
    # Exclude orders matching the comment pattern
    if not excluded {
        self.cust_order_count[here.o_custkey] += 1;
    }
}
```

Customers with no qualifying orders retain their zero count, preserving the LEFT JOIN semantics.

5 Data Loading and Verification

5.1 Data Generation

We use the TPC-H `dbgen` tool to produce pipe-delimited `.tbl` files at scale factor 0.01. This yields approximately 86,800 nodes across all eight table types.

5.2 Graph Construction

A dedicated loader module (`loader.jac`) reads each `.tbl` file, instantiates typed nodes, and connects them via typed edges using lookup maps keyed by primary keys. For example, each NATION row is connected to its parent REGION via a `region_has_nation` edge:

```
n = region_map[regionkey]
    +>: region_has_nation() :+> nation_node;
```

Loading the full SF = 0.01 dataset takes approximately 4–5 minutes.

5.3 Reference Answer Generation

Correctness verification requires reference answers. We load the same `.tbl` files into an in-memory SQLite database and execute the official TPC-H SQL queries. The results are serialized to pipe-delimited `.out` files (one per query), which serve as ground truth.

5.4 Verification

After running all 22 walker queries, a verification script (`verify_answers.py`) compares each query's output against the reference `.out` file, checking row counts, column values, and sort order with tolerance for floating-point rounding. All 22 queries pass at SF = 0.01.

6 Performance Observations

While this work focuses on expressiveness rather than performance, we note several observations at SF = 0.01.

The 22 queries fall into three runtime tiers: *fast* queries (Q2, Q7, Q8, Q11, Q16, Q17, Q22) complete in 0.3–3 seconds; *medium* queries (Q3, Q5, Q10, Q13) require 1–6 seconds; and *lineitem-traversal* queries (Q1, Q4, Q6, Q9, Q12, Q14, Q15) take approximately 12 seconds each, as they visit all ∼60,000 line-item nodes.

The Jac walker runtime is implemented on top of Python and uses recursive function calls for graph traversal. At SF = 0.01, the traversal depth can exceed Python's default recursion limit,

requiring it to be raised to 500,000 or higher. This limits scalability to larger scale factors without runtime modifications.

The multi-hop traversal syntax provides a significant speedup over the single-hop alternative. In the single-hop version, separate entry abilities fire at every intermediate node type, incurring Python function-call overhead at each of the ∼86,000 nodes. Multi-hop traversal skips intermediate abilities entirely, visiting only the target nodes.

7 Discussion

Traversal vs. Joins. Graph traversal is a fundamentally different computation model from relational joins. In SQL, a five-table join is a single declarative statement that the optimizer can reorder. In Jac, the same operation is an explicit path through the graph, with the programmer controlling traversal order. This gives more control but shifts optimization responsibility to the developer.

Walker State as Hash Tables. Many TPC-H queries require combining data from multiple tables. In the walker model, this is achieved by accumulating data into `has` variables (typically dictionaries) as the walker traverses intermediate nodes, then using that state at deeper levels. This is functionally equivalent to building hash tables for hash-join execution in relational engines.

The Helper Walker Pattern as Subquery Execution. Spawning helper walkers to build lookup maps before the main traversal mirrors how relational engines execute correlated subqueries by materializing intermediate results. Jac makes this explicit: the programmer decides what to pre-compute and what to compute inline.

Limitations. The current implementation has several limitations: (1) Every query traverses from `root`, as the language restricts traversal origins to `here` (the current node) rather than arbitrary variable references; (2) there are no secondary indexes, so selective queries must still visit all nodes along the path; (3) Python's recursion limit constrains the graph size; and (4) the persistent graph database accumulates data across runs and must be manually cleared.

8 Conclusion

We have demonstrated that all 22 TPC-H queries can be faithfully expressed as graph walker traversals in Jac, a data-spatial programming language. The multi-hop traversal syntax reduces per-query implementation complexity from five or more entry abilities to one or two, while maintaining correctness against SQL reference answers.

Five recurring patterns—direct multi-hop aggregation, filtered intermediate traversal, helper walkers, two-phase traversal, and LEFT JOIN emulation—cover the full range of TPC-H query types. These patterns suggest that graph-native languages can serve as a viable alternative paradigm for expressing analytical workloads, though performance optimization and scalability remain open challenges.

Future work includes testing at larger scale factors, comparing performance with established graph databases, and exploring language-level optimizations such as parallel walker execution and automatic index generation.

References

[1] Transaction Processing Performance Council, "TPC-H Benchmark Specification," Revision 3.0.1, 2022. https://www.tpc.org/tpch/

[2] J. Jaseci, "Jac Language Documentation," 2024. https://www.jac-lang.org/

[3] Y. Mars *et al.*, "Jaseci: A Data-Spatial Programming Framework," 2023. https://github.com/Jaseci-Labs/jaseci